UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT ENTITLED

DESIGN AND IMPLEMENTATION OF A KNOWLEDGE GRAPH BASED MULTI-AGENT ORCHESTRATION FRAMEWORK

BY

KASSER-TEE ATHANASIUS YINTEMANN, EMILY WOBI

EDEM SOKATSI, NUGAH SELASE MARK

NOAH OTINKORANG BOYE

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

PROJECT SUPERVISOR

.....

Mr. Frederick E. Broni

Tarkwa, Ghana

March, 2026

DECLARATION

We, ..., declare that this project work is our own original work. It is being submitted for the degree of Bachelor of Science in Computer Science and Engineering at the University of Mines and Technology (UMaT), Tarkwa. It has not been submitted, either in whole or in part, for any degree or examination at any other university.

Signature of Student: _____

[Sokatsi Edem] (FOE.41.008.142.22)

Signature of Student: _____

[Nugah Selase Mark] (FOE.41.008.111.22)

Signature of Student: _____

[Wobi Emily Wepiah] (FOE.41.008.152.22)

Signature of Student: _____

[Kasser-Tee Athanasius Yintemann] (FOE.41.008.087.22)

Signature of Student: _____

Noah Otinkorang Boye (FOE.41.008.063.22)

_____ day of _____ (year) _____

ABSTRACT

Current LLM Based Multi-Agent system.....

DEDICATION

I dedicate....

ACKNOWLEDGMENTS

I attribute....

TABLE OF CONTENTS

DECLARATION **i**

ABSTRACT **ii**

DEDICATION **iii**

ACKNOWLEDGMENTS **iv**

TABLE OF CONTENTS **v**

LIST OF FIGURES **viii**

LIST OF TABLES **ix**

LIST OF ABBREVIATIONS **x**

CHAPTER 1 INTRODUCTION **1**

1.1 Background Study 1

1.2 Problem Statement 3

1.3 Aim of the Study 4

1.4 Objectives 5

1.5 Methods Used 5

1.6 Tools and Facilities 6

1.6.1 Hardware 6

1.6.2 Software 6

1.7 Scope 6

1.8 Work Organization 6

CHAPTER 2 LITERATURE REVIEW **8**

2.1 Introduction 8
2.2 Introduction to Concepts 10
2.2.1 LLM-Based Agents 10
2.2.2 Multi-Agent Systems 12
2.2.3 Knowledge Graphs 13
2.3 Review of Related Works 15
2.3.1 Multi-Agent Orchestration 15
2.3.2 Shift Towards Structured But Dynamic Orchestration 19
2.3.3 Extending KG Driven Discovery to MAS 20
2.4 Conclusion 21

CHAPTER 3 METHODOLOGY AND SYSTEM DESIGN 22
3.1 Introduction 22
3.2 Research Design 22
3.3 System Architecture 23
3.4 Knowledge Graph Design 24
3.4.1 Graph Structure: Nodes 25
3.4.2 Graph Structure: Relationships (Edges) 26
3.5 Agent Design 26
3.5.1 Agent Roles and Capabilities 27
3.5.2 Agent Interaction Model 27
3.6 Orchestration Mechanism 28
3.6.1 Task Allocation and Decision Flow 28
3.7 Implementation Tools and Technologies 29
3.8 System Workflow 30
3.9 System Evaluation and Testing 30
3.9.1 Evaluation Metrics 32
3.9.2 Testing Procedures 32

CHAPTER 4 CHAPTER TITLE HERE 34
4.1 FIRST SECTION TITLE GOES HERE 34
4.1.1 Your subsection 34

CHAPTER 5 CONCLUSION AND RECOMMENDATION 35

5.1 Conclusion 35
5.2 Recommendations 35

REFERENCES **36**

LIST OF FIGURES

Figure	Title	Page
1.1	Recent trends in LLM-based multi-agent research by category and paper count (Guo *et al.*, 2024).	1
2.1	Modular architecture of an LLM-based agent showing the interaction between the reasoning core, memory, and tools (Authors' own construct).	10
2.2	Example Multi-Agent System (MAS) architecture (Yang *et al.*, 2024).	12
2.3	Empirical performance of agent scaling across architectures, showing average benchmark performance versus model Intelligence Index for different LLM families and coordination strategies (adapted from (Kim *et al.*, 2025)).	17
3.1	Overall system architecture showing the interaction between the User CLI, Orchestrator, Planner, Executor, Reasoning Engine, and Neo4j Knowledge Graph (Authors' own construct).	25
3.2	System workflow flowchart showing the sequence from system boot through the execution loop to result delivery (Authors' own construct).	31

LIST OF TABLES

Table	Title	Page
2.1	Mapping of Multi-Agent patterns to theoretical classifications.	18
3.1	Knowledge Graph Node Types	25
3.2	Knowledge Graph Relationship Types	26
3.3	Development Tools and Technologies	29
3.4	System Evaluation Metrics	32

LIST OF ABBREVIATIONS

DAG	Direct Acyclic Graph
KG	Knowledge Graph
AI	Artificial Intelligence
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
MAS	Multi-Agent System
API	Application Programming Interface
TOT	Tree of Thoughts
CoT	Chain of Thought
ReAct	Reasoning and Acting
RDF	Resource Description Framework
LPG	Labeled Property Graph
SAS	Single Agent System
API	Application Programming Interface

CHAPTER 1

INTRODUCTION

1.1 Background Study

Large Language Model (LLM)-based multi-agent systems, where several LLM-powered agents collaborate to accomplish tasks that would otherwise be difficult for a single agent, have demonstrated significant potential in automating complex procedures within business workflows. These systems leverage the reasoning, planning, and language understanding capabilities of LLMs while distributing responsibilities across specialized agents (Weng, 2023). As a result, multi-agent architectures have emerged as a promising paradigm for building intelligent systems capable of handling multi-step and domain-specific tasks.

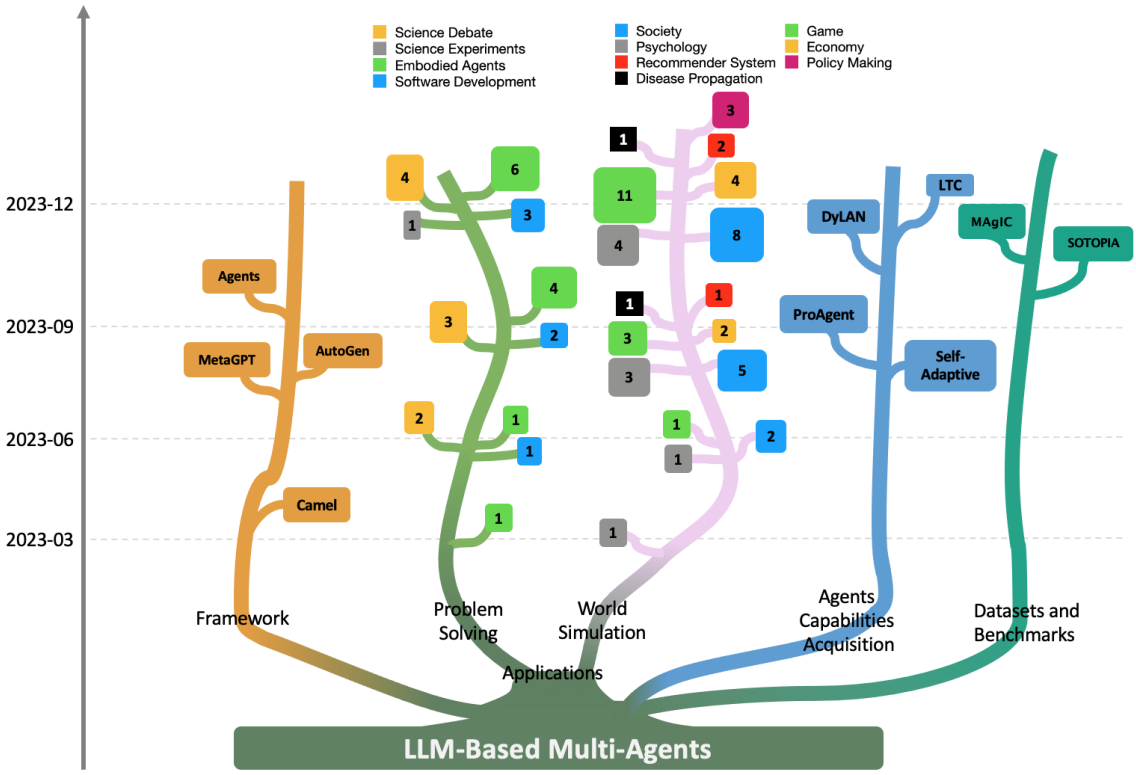

Figure 1.1 Recent trends in LLM-based multi-agent research by category and paper count (Guo *et al.*, 2024).

Recent research indicates a growing interest in agentic AI frameworks such as LangGraph, CrewAI, and AutoGen. These frameworks provide architectural foundations and tooling for designing, orchestrating, and deploying intelligent agent systems (Derouiche *et al.*, 2025).

They enable developers to define agents with specialized roles, establish communication mechanisms between agents, and coordinate task execution across multiple components. According toTran *et al.* (2025), current multi-agent collaboration mechanisms can be categorized across several dimensions. Collaboration types include cooperation, competition, and coopetition, describing how agents interact to achieve their objectives. Collaboration strategies may be rule-based, role-based, or model-based, depending on how agent behavior is defined and coordinated. Communication structures may follow centralized, decentralized, or hierarchical patterns, while orchestration architectures may be static or dynamic depending on the nature of the task and domain requirements. These design principles collectively describe the operational foundations of modern multi-agent frameworks such as LangGraph, CrewAI, and AutoGen.

In practice, frameworks such as LangGraph and AutoGen often recommend the use of Directed Acyclic Graphs (DAGs) to model workflows that require strict sequencing and deterministic execution. This approach is particularly common in enterprise and business workflows, where the order of task execution must be explicitly defined to reduce uncertainty arising from the probabilistic nature of LLMs. However, DAG-based workflow definitions require developers to manually encode the sequence of tasks in advance. This introduces limitations in scenarios where workflows must adapt dynamically based on task context, domain knowledge, or intermediate outcomes.

Knowledge graphs provide a complementary approach to addressing this challenge. Knowledge graphs represent information using semantic relationships between entities, enabling datasets to be connected through their underlying meaning rather than purely structural links Hogan *et al.* (2021). By representing entities, relationships, and constraints within a domain, knowledge graphs offer a structured and extensible way to model domain knowledge. Recent research Nizar *et al.* (2025), Sanmartin (2024), and Kau *et al.* (2024) etc. have demonstrated the potential of integrating knowledge graphs with LLM-based systems. For example, knowledge graphs have been used to enhance retrieval-augmented generation, such as in KG-RAG (Sanmartin, 2024), to improve tool and agent retrieval mechanisms (Nizar *et al.*, 2025), and to ground LLM reasoning in structured knowledge representations (Amayuelas *et al.*, 2025). Other applications include knowledge injection, semantic reasoning, and improving contextual understanding in LLM systems (Kau *et al.*, 2024).

Building on these developments, this project aims to design and implement a knowledge graph based multi-agent orchestration framework that leverages knowledge graphs to dynamically infer task sequencing within a workflow. Unlike traditional approaches that require explicit workflow definitions, the proposed framework relies on the knowledge graph to provide domain grounding, guide task decomposition, and facilitate tool and agent delegation. By leveraging semantic relationships between tasks, tools, and domain entities, the system can dynamically determine appropriate execution paths for a given workflow.

1.2 Problem Statement

Enterprise and business workflows typically require clearly defined task sequencing and explicit role delegation in order to ensure reliable and predictable execution Elementum Team (2026). In multi-agent systems, this requirement is commonly addressed using structured workflow representations such as Directed Acyclic Graphs (DAGs), where nodes represent tasks or agents and edges define dependencies that determine execution order Nath (2025).

Modern agent orchestration frameworks such as LangGraph often adopt DAG-based workflow structures to model task dependencies and ensure deterministic execution paths. DAGs enable systems to enforce task ordering, avoid cyclic dependencies, and manage complex workflows by representing tasks as nodes connected through directed edges LangChain Inc. (2024). This structure has been widely adopted in workflow orchestration and pipeline management systems due to its simplicity and reliability in representing sequential processes (Shankar, 2025).

While DAG-based orchestration is well-suited to structured workflows, it may present challenges in larger systems supporting many variations of agent–tool combinations. For example, one might anticipate the need to define and maintain separate DAGs for different workflows, which could introduce additional complexity. The process of designing, updating, and managing these DAG structures becomes increasingly complex and difficult to scale as the number of workflows increases. Furthermore, in cases where new agents or tools need to be introduced into the system, existing DAGs often need to be reconstructed or modified to incorporate these new components. This requirement increases the maintenance burden and reduces system flexibility, particularly in dynamic environments where agents and tools evolve over time. This limitation creates a critical gap in current multi-agent orchestration

approaches: workflow structures must be explicitly defined rather than dynamically inferred. As a result, systems lack the ability to automatically determine appropriate task sequencing based on contextual knowledge of available agents, tools, and domain constraints.

Consequently, an important research question arises: Can workflow structures be automatically generated instead of explicitly defined? Furthermore, how much contextual or domain knowledge is required for a system to infer appropriate task sequencing and agent delegation dynamically?

This research addresses this gap by proposing a Knowledge Graph (KG)-based multi-agent orchestration framework that automatically infers workflows by leveraging semantic relationships within a knowledge graph. Instead of relying on predefined DAGs, the proposed framework models relationships between tasks, agents, and tools within a knowledge graph and determines appropriate execution sequences by traversing this structured representation of domain knowledge. In addition, the proposed solution supports dynamic agent discovery, allowing new agents or tools to be incorporated into the system without requiring reconstruction of existing workflows. By properly linking newly introduced agents or tools to the knowledge graph through defined semantic relationships, they immediately become part of the orchestration ecosystem and can be discovered and utilized during workflow inference.

Through this approach, the framework aims to provide a scalable, adaptive, and extensible orchestration mechanism for multi-agent systems, reducing the need for manually defined workflows while enabling flexible integration of new agents and tools.

1.3 Aim of the Study

The aim of this research is to propose a knowledge graph–based multi-agent orchestration framework that dynamically infers workflow structures in LLM-based multi-agent systems. The framework leverages semantic relationships within a knowledge graph to guide task sequencing, agent selection, and tool delegation without requiring explicitly predefined workflows such as Directed Acyclic Graphs (DAGs).

1.4 Objectives

- To design a knowledge graph–based multi-agent orchestration framework, including task representation, agent modeling, and a traversal strategy for dynamic task sequencing and agent/tool selection.
- To develop an orchestrator agent to coordinate agents and manage workflow execution within the proposed framework.
- To implement and evaluate a prototype application to assess the framework's effectiveness in dynamically generating workflows and supporting multi-agent interactions.

1.5 Methods Used

This research follows a Systems-Oriented Research Methodology, focusing on the architectural design, implementation, and empirical evaluation of a novel orchestration engine designed to dynamically infer workflows in LLM-based multi-agent systems using Knowledge Graphs.

- A conceptual architecture for the knowledge graph–based orchestration framework is designed, defining the structure for task entry, agent representation, task delegation, and task completion.
- A knowledge graph schema is developed to represent the relationships between tasks, agents, and tools. This structure provides the semantic foundation required for workflow inference.
- An orchestrator agent is implemented to coordinate task execution, interact with the knowledge graph, and delegate tasks to appropriate agents or tools.
- A traversal mechanism is designed to enable the system to navigate relationships within the knowledge graph in order to infer task sequencing and identify suitable agents or tools for execution.
- A prototype system implementing the proposed framework is developed to demonstrate its functionality and validate the design.

- The framework is evaluated using selected task scenarios to assess its ability to dynamically generate workflows, coordinate agents, and integrate additional agents or tools within the system.

1.6 Tools and Facilities

1.6.1 Hardware

- MacBook Air M2, 16GB RAM / MacBook Air M3, 16GB RAM

1.6.2 Software

- Foundational LLM model via API or local deployment
- Knowledge Graph database (Neo4j)
- Python 3.10+
- Vector database (chromadb)

1.7 Scope

This research focuses on the design and implementation of a knowledge graph–based multi-agent orchestration framework for LLM-powered agent systems. The study covers the design of the framework architecture, development of an orchestrator agent, modeling of a knowledge graph to represent tasks, agents, and tools, and the implementation of a traversal strategy to enable dynamic workflow inference. A prototype application will be developed to demonstrate the functionality of the framework and evaluate its ability to coordinate agents and generate workflows dynamically. The scope of this research does not include the training or evaluation of large language models, as the study focuses on the orchestration framework rather than improvements to the underlying LLM models.

1.8 Work Organization

This project work is organized into five chapters. Chapter One deals with the introduction establishing context and goals. Chapter Two focuses on literature review, analyzing current approaches to Multi-Agent Orchestration and application of KG to ground LLM reasoning.

Chapter Three details the methodology and KG Orchestration Framework design and implementation. Chapter Four system testing and discussion with comprehensive evaluation results. Chapter Five provides conclusion and recommendations, summarizing findings and future work.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

The evolution of Large Language Models (LLMs) has transcended the boundaries of simple linguistic synthesis, moving toward the realization of agentic systems capable of autonomous, multi-step task execution (Jian, 2025). While the generative prowess of these models is undeniable, their deployment as singular autonomous entities has exposed significant intrinsic limitations, most notably the propensity for hallucination and the constraints of their autoregressive nature (Tran *et al.*, 2025). These models often struggle with "slow-thinking" processes (Hagendorff *et al.*, 2023) the systematic, deliberative reasoning required for complex problem-solving and face diminishing returns as scaling laws meet the reality of open-ended computational logic (Kaplan *et al.*, 2020). To bridge these gaps, most implementations of agentic applications have pivoted toward Multi-Agent Systems (MAS), under the observation that a collective of specialized agents consistently outperforms individual models by distributing cognitive load and introducing cross-verification (Albada, 2025a). This shift, however, necessitates a rigorous definition of how these agents collaborate, communicate, and are orchestrated to ensure that the collective output remains coherent and goal-oriented.

Tran *et al.* (2025) describes orchestration in the context of MAS as a multifaceted discipline encompassing collaboration types, coordination strategies, and specific communication structures designed to manage the flow of information between disparate agents. A primary focus within this field is the "sequencing" of agent interventions, which dictates the operational logic of the workflow. In many current frameworks, this sequencing is either statically defined through manual scripting or relies on heuristic-based "context hand-offs," where the state of a task is passed linearly from one agent to the next based on probabilistic triggers. Such strategies, while functional for experimental or simple workflows, frequently fail to meet the reliability benchmarks required of enterprise systems, where reliability is defined by the consistency and accuracy of generated outputs (Menshawy and Fahmy, 2025). Because enterprise deployments demand that reliability be maintained through rigorous testing, validation phases, and robust version control, the inherent "stochastic guessing" of

probabilistic sequencing introduces unacceptable inaccuracies. The reliance on models that may be compromised by training biases or learning anomalies necessitates a move toward orchestration that does not just "predict" the next step, but validates it against a quality assurance framework ensuring that every agentic transition is as dependable as a traditional enterprise software deployment.

To anchor these agentic decisions in reality, researchers have historically looked to Knowledge Graphs (KGs) as a robust mechanism for grounding LLM outputs (Kau *et al.*, 2024). Knowledge Graphs provide a structured, symbolic representation of a domain, offering a "Source of Truth" and a relational depth that probabilistic models inherently lack (Hogan *et al.*, 2021). By integrating KGs into the decision-making loop, systems can verify agent claims against a persistent, factual schema, thereby mitigating the risks of stochastic drift and ensuring semantic consistency. The intersection of graph-based reasoning and agent orchestration represents a significant frontier in AI research, as it promises to transform the way agents understand their environment and their own functional capabilities. This historical precedent for KG-driven grounding suggests that the next logical step in the evolution of MAS is the use of graph structures not just for static data retrieval, but as the fundamental logic gate for orchestration itself.

This chapter provides a comprehensive examination of existing approaches to multi-agent orchestration, tracing the trajectory from general collaboration frameworks to specialized strategies focused on agent sequencing and context hand-off, as well as emerging approaches that propose dynamic methods for managing these transitions. By critically analyzing these methodologies, the review highlights a persistent limitation: the lack of a truly dynamic and deterministic way to decide the order of execution in complex, non-linear workflows. Many current "dynamic" models still operate within the vacuum of the LLM's context window, leaving the sequencing vulnerable to the same autoregressive failures the MAS architecture was intended to solve. Through this analysis, we establish the critical need for a fact-grounded approach to dynamic agent sequencing. This leads directly to the core contribution of this project: a Knowledge Graph-Based Multi-Agent Orchestration Framework that utilizes a deterministic world model to ensure that every agentic transition is logically validated and anchored in the objective reality of the graph.

2.2 Introduction to Concepts

This section provides comprehensive explanations of the key concepts from LLM-Based Agents, Multi-Agent Systems to Knowledge Graphs, ensuring a solid theoretical foundation.

2.2.1 LLM-Based Agents

To define the foundational components of the framework, it is first necessary to establish the architectural anatomy of an LLM-Based Agent. Unlike a standard language model, which functions as a passive text generator, an agent is a semi-autonomous system that uses the LLM as its "reasoning engine" to interact with an external environment. The literature Xi *et al.* (2023) generally decomposes an agent into three primary pillars Memory, Planning, and Tool Use often abstracted as the "brain," "perception," and "action" of the system.

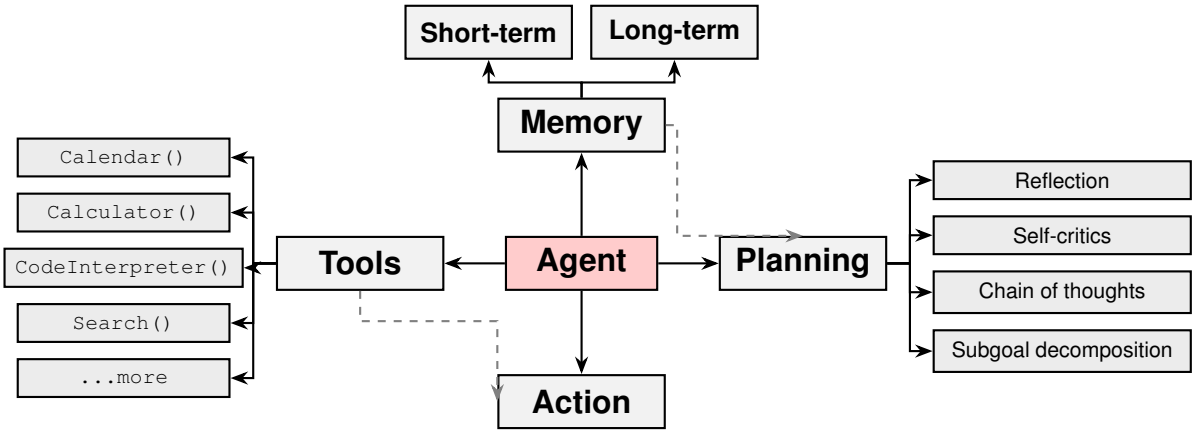

Figure 2.1 Modular architecture of an LLM-based agent showing the interaction between the reasoning core, memory, and tools (Authors' own construct).

Memory allows the agent to maintain state over time, categorizing information into short-term storage (the context window) and long-term storage (external databases or Retrieval-Augmented Generation) (Albada, 2025b).

Tool use (Schick *et al.*, 2023; Parisi *et al.*, 2022) enables the agent to transcend the "digital silo" of its training data by invoking external APIs, executing code, or querying databases to retrieve real-time information.

However, the most critical and most volatile component is the Planning layer, which governs how the agent cognitively breaks down a complex objective into executable steps. The current

state-of-the-art in agentic planning relies almost exclusively on prompt-based techniques designed to elicit "System 2" thinking deliberative, analytical reasoning from the model. Strategies such as Chain-of-Thought (CoT) force the model to generate intermediate reasoning steps (Wei *et al.*, 2023), while Tree-of-Thoughts (ToT) allows the agent to explore multiple reasoning branches and self-evaluate their progress in a non-linear fashion (Yao *et al.*, 2023a). More advanced frameworks like ReAct (Reasoning and Acting) create a feedback loop where the model interleaves thoughts, actions, and observations to refine its trajectory (Yao *et al.*, 2023b).

While these methods significantly improve performance on isolated tasks, they remain purely probabilistic and heuristic. The planning is "soft-coded" within the prompt, meaning the agent's logic is only as reliable as the model's next-token prediction. There are no hard constraints to prevent the agent from skipping a vital step or proposing a logically impossible action, as the "plan" is essentially a sophisticated guess based on linguistic patterns rather than a validated structural map.

In terms of Task Execution, LLM-based agents typically operate through either direct handling or tool invocation. In the former, the LLM processes information and generates a response based on its internal weights; in the latter, the agent must translate its intent into a structured output (Liu *et al.*, 2024), such as a JSON-formatted function call, to interact with an execution tool. The tool returns a raw response, which the LLM must then re-process into a natural language summary or use as a catalyst for the next step in the plan. This execution phase is where the "grounding" of the agent is tested, as it requires the model to precisely map its internal reasoning to the strict syntactic requirements of external software. Any misalignment between the LLM's generative output and the tool's expected schema results in execution failure, highlighting the fragility of relying on linguistic intuition for technical interoperability.

Despite the sophistication of these components, single-agent systems frequently succumb to catastrophic failure when applied to complex, multi-stage enterprise workflows. These failures are primarily attributed to the "unconstrained" nature of prompt-based planning (Rana and Kumar, 2025). Without an external governing structure, a single agent often enters a hallucination recursive loop, where it attempts to correct an initial error by generating further inaccurate logic based on that false premise (Latitude, 2024; Cheng *et al.*, 2025). Additionally, as the task complexity increases, the agent suffers from contextual overload; the

accumulation of previous thoughts, tool outputs, and observations clutters the context window, leading to a precipitous degradation in reasoning quality (Anthropic, 2024). This architectural fragility underscores the necessity for moving away from isolated, prompt-driven agents toward a more rigid, orchestrated multi-agent environment where the "world model" is held by a Knowledge Graph rather than the volatile and finite memory of a single model.

2.2.2 Multi-Agent Systems

The evolution of agentic architectures from single-model entities to Multi-Agent Systems (MAS) represents a strategic transition toward functional decomposition and collaborative problem-solving. In a MAS environment, a complex global objective is partitioned into discrete sub-tasks, each managed by specialized agents possessing distinct roles, personas, and toolsets (Yang *et al.*, 2024). This shift is predicated on the observation that the collective intelligence of a modular system consistently outperforms the reasoning capacity of a monolithic LLM, particularly in tasks requiring high cognitive diversity or parallel execution (Li *et al.*, 2024; Guo *et al.*, 2024). By isolating specific domains of expertise such as an agent dedicated solely to code generation and another to quality assurance the system introduces a layer of intersubjective verifiability, where agents can critique and refine each other's outputs, effectively serving as a circuit breaker for the "hallucination recursive loops" prevalent in single-model configurations.

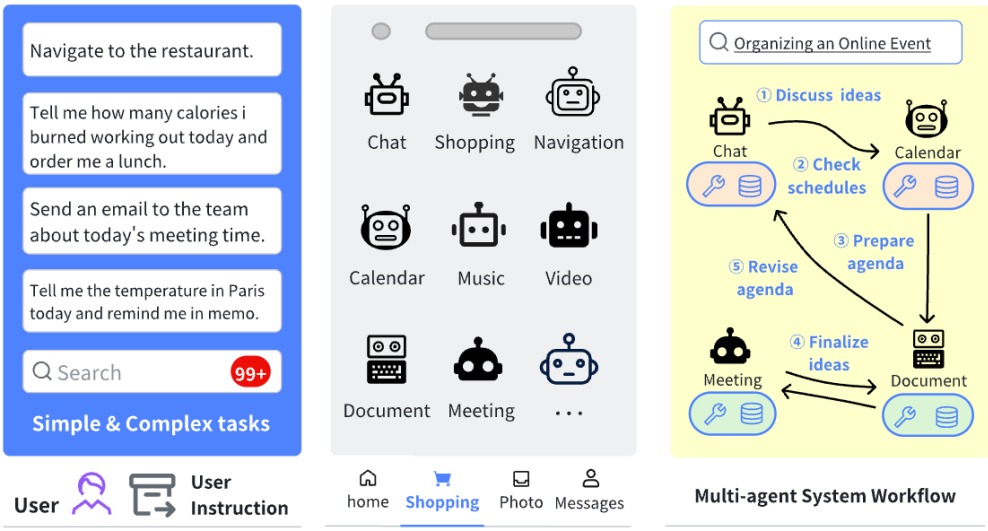

Figure 2.2 Example Multi-Agent System (MAS) architecture (Yang *et al.*, 2024).

Despite these advantages, the transition to MAS introduces a significant coordination overhead, commonly referred to as the “Coordination Tax” (Kim *et al.*, 2025), as the number of agents increases, the complexity of managing their interactions grows exponentially, necessitating sophisticated communication protocols and collaboration strategies. Wang *et al.* (2025) distinguishes between "Horizontal" collaboration, where agents interact as peers in an unstructured manner, and "Vertical" or "Hierarchical" orchestration, where a lead agent or central controller dictates the workflow. Without a formal governing structure, even these collaborative models remain bound by the limitations of the underlying models’ ability to maintain a consistent state across fragmented context windows.

Consequently, Valmeekam *et al.* (2023) underscores that while Large Language Models (LLMs) can generate text that mimics the structure of a plan, they consistently fail at logically validated execution. This empirical "planning gap" demonstrates that agent collaboration and orchestration cannot be left to the probabilistic nature of the LLM alone. By shifting the responsibility of workflow logic from the LLM to a structured world model, the framework ensures that every agentic action is not just linguistically plausible, but mathematically verified and fully auditable. This transition effectively moves the system from a paradigm of "stochastic sequencing" to one of "deterministic coordination," providing the reliability required for enterprise-grade applications.

2.2.3 Knowledge Graphs

The final component in this theoretical foundation is the Knowledge Graph (KG), a multi-relational graph consisting of entities as nodes and their interactions as edges (Hogan *et al.*, 2021). Historically, Knowledge Graphs emerged from the Semantic Web movement and classical symbolic AI, where they were used to provide a structured, machine-readable representation of world knowledge independent of natural language processing. Outside LLMs, KGs are utilized for a variety of applications such as expert systems (Jiang *et al.*, 2025), recommendation engines (Catherine and Cohen, 2016), and data integration (Noy *et al.*, 2019), providing a deterministic framework for representing facts in a $G = (V, E, R)$ structure, where V is a set of vertices (entities), R is a set of relation types, and $E \subseteq V \times R \times V$ represents the directed, labeled edges or "triples."

(Kejriwal *et al.*, 2021) highlights two primary models that dominate the architectural

landscape: Resource Description Framework (RDF) and Labeled Property Graphs (LPG). RDF is a standardized model for data interchange that treats every statement as a "triple" $(subject, predicate, object)$. It is highly interoperable and often used for web-scale data integration using the SPARQL query language. Conversely, Property Graphs allow for a more flexible, internal data structure where both nodes and edges can contain "properties" (key-value pairs). While RDF is rooted in formal logic and global URI standards, Property Graphs are optimized for high-performance traversal and are frequently used in enterprise settings where complex, interconnected metadata must be queried efficiently using languages like Cypher.

The operational utility of a KG is realized through Graph Databases and their associated Traversal Algorithms. Unlike relational databases that rely on expensive "JOIN" operations, graph databases use index-free adjacency, meaning each node effectively points to its neighbors (Stegeman, 2023). This allows for complex pathfinding and semantic traversal, which can be mathematically expressed as finding a sequence of nodes $v_0, v_1, \ldots, v_n$ such that $(v_i, r, v_{i+1}) \in E$ for each i. This capability is crucial for identifying functional dependencies; if a goal state S_g is reachable from a current state S_c, a graph traversal algorithm can determine the optimal sequence of agentic transitions $a_1, a_2, \ldots, a_k$ required to bridge the gap, ensuring that each step satisfies the pre-conditions defined in the graph.

The integration of Large Language Models (LLMs) and Knowledge Graphs (KGs) has evolved into a bidirectional synergy designed to address the inherent limitations of both technologies, primarily by bridging the gap between probabilistic linguistic intuition and deterministic symbolic logic. As highlighted in foundational surveys Pan *et al.* (2024) and Kau *et al.* (2024), this intersection is typically categorized into three paradigms: KG-enhanced LLMs, where structured facts ground generative outputs to mitigate hallucinations; LLM-enhanced KGs, where models automate the labor-intensive tasks of entity extraction and relationship prediction; and Synergized LLM + KG frameworks, where both components interact in a continuous neuro-symbolic feedback loop. Practical applications of these paradigms have demonstrated significant improvements in system reliability; for instance, KG-RAG (Sanmartin, 2024) utilizes the topological structure of a graph to perform multi-hop reasoning that exceeds the capabilities of standard vector-based retrieval, while further research explores the use of KGs to refine tool and agent retrieval mechanisms (Nizar *et al.*, 2025), ensuring

that the selection of external functions is grounded in a structured understanding of their functional dependencies and semantic constraints. Specialized techniques continue to refine this integration, such as the "Chain-of-Knowledge" (CoK) framework (Zhang *et al.*, 2024a), which allows LLMs to learn structured reasoning paths directly from KGs to internalize symbolic logic, and the "Self-Refinement-Enhanced Knowledge Retrieval" approach, which introduces an iterative loop where the LLM critiques its own retrieved facts against a KG to eliminate contradictions (Niu *et al.*, 2024). Additionally, advancements in "Question-guided KG Re-scoring and Injection" (Zhang *et al.*, 2024b) optimize Question Answering (KGQA) by dynamically re-weighting the relevance of graph nodes based on the specific linguistic context of the query, collectively highlighting a shift from passive data retrieval toward active, structural reasoning where the Knowledge Graph serves as the definitive logical anchor for generative AI.

2.3 Review of Related Works

This section reviews the evolution of Multi-Agent Collaboration, focusing on the transition from heuristic-driven interactions to formal Structured Orchestration Mechanisms. It narrows the analysis to Dynamic Structured Orchestration, where adaptive coordination is achieved in complex, non-linear environments. These works highlight the "planning gap" in current systems and the necessity for the factually grounded architectures proposed in this framework.

2.3.1 Multi-Agent Orchestration

Let's begin by adopting multi-agent system formalism from Guo *et al.* (2024) and Zhu *et al.* (2025). An agentic system $\mathcal{S}$ is defined as a four-tuple $\mathcal{S} = \langle \mathcal{A}, \mathcal{E}, \mathcal{C}, \Omega \rangle$, where $\mathcal{A} = \{a_1, \ldots, a_n\}$ is a set of $n \geq 1$ agents, $\mathcal{E}$ is the shared environment, $\mathcal{C}$ is the communication topology, and Ω is the orchestration policy. Systems where $|\mathcal{A}| = 1$ are Single-Agent Systems (SAS), while $|\mathcal{A}| > 1$ are Multi-Agent Systems (MAS). Each agent $a_i \in \mathcal{A}$ is a tuple $a_i = \langle \Phi_i, \mathcal{X}_i, \mathcal{M}_i, \pi_i \rangle$, consisting of a reasoning policy Φ_i (typically an LLM), an action space $\mathcal{X}_i = \{\text{ToolCall}(t, \theta) : t \in \mathcal{T}, \theta \in \Theta_t\}$, internal memory $\mathcal{M}_i$, and a decision function $\pi_i : \mathcal{H} \to \mathcal{X}_i$ mapping history $\mathcal{H}$ to actions. At timestep t, the execution logic follows:

$$\alpha_{i,t} = \pi_i(h_{i,t})$$

$$o_{i,t} = \mathcal{E}(\alpha_{i,t})$$

$$h_{i,t+1} = h_{i,t} \oplus (\alpha_{i,t}, o_{i,t})$$

The history $h_{i,t}$ is initialized as $\{s_0\}$ and updated iteratively, subject to a maximum context constraint $|h_{i,t+1}| \leq K$. Communication in MAS is facilitated by Ω through message passing between agents.

To categorize the structural variations in multi-agent coordination, Kim *et al.* (2025) establishes a formal distinction between communication (the mechanical passing of messages) and coordination (the strategic direction of agent activities). This distinction is operationalized through the definition of a communication topology C, which governs the information flow patterns within an agent system $\mathcal{S}$. The paper identifies four primary topological configurations:

- **Independent** ($C = \emptyset$): Characterized by a total absence of inter-agent communication, where agents operate in isolation.
- **Centralized** ($C = \{(a_{\text{orch}}, a_i) : \forall i\}$): Information flow is strictly restricted to an orchestrator-to-agent star pattern.
- **Decentralized** ($C = \{(a_i, a_j) : \forall i, j, i \neq j\}$): An all-to-all topology allowing for peer-to-peer information fusion.
- **Hybrid** ($C = C_{\text{centralized}} \cup C_{\text{peer}}$): A composite structure that combines hierarchical oversight with limited lateral peer-to-peer exchange.

These topologies lead to four specific MAS Configurations, each presenting unique trade-offs between parallelization, coordination depth, and computational overhead. An Independent MAS ($A = \{a_1, \ldots, a_n\}, \Omega = synthesis_only$) maximizes parallelization but offers minimal coordination, making it suitable for ensemble-style reasoning with a complexity of $O(nk)$. Conversely, a Centralized MAS ($A = \{a_{orch}, a_1, \ldots, a_n\}, \Omega = hierarchical$) stabilizes reasoning through hierarchical verification but introduces a central bottleneck, scaling at $O(rnk)$ across r coordination rounds. In a Decentralized MAS ($A = \{a_1, \ldots, a_n\}, \Omega = consensus$), agents engage in d sequential debate rounds to reach a collective agreement, incurring a memory complexity of $O(dnk)$ as each agent maintains its own debate history. Finally, the Hybrid MAS ($\Omega = hierarchical + lateral$) attempts to inherit the control of a central

orchestrator while maintaining the flexibility of lateral peer exchanges, represented by the complexity $O(rnk+pn)$. By formalizing these architectures, the research enables a structural ablation of coordination mechanics, allowing performance gains to be attributed to specific topological designs rather than generic multi-agent effects.

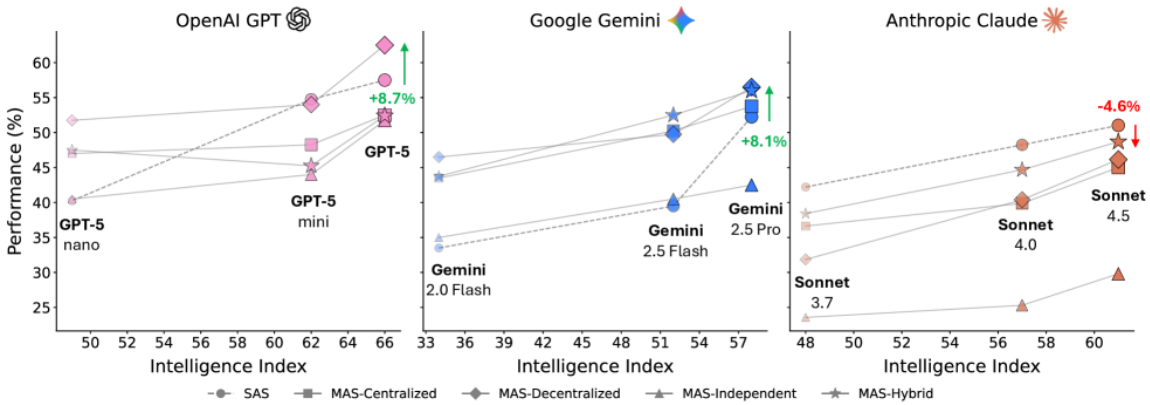

Figure 2.3 Empirical performance of agent scaling across architectures, showing average benchmark performance versus model Intelligence Index for different LLM families and coordination strategies (adapted from (Kim *et al.*, 2025)).

Tran *et al.* (2025) also identifies a multi-dimensional taxonomy consisting of Collaboration Types (Cooperation, Competition, Coopetition), Strategies (Rule-based, Role-based, Model-based),Communication Structures (Centralized, Decentralized, Hierarchical) and Coordination and Orchestration (Static , Dynamic Architectures). However, the primary focus of this project is on Coordination and Orchestration, which is defined as how collaboration channels are created, ordered, and characterized, effectively forming the backbone of multi-agent interactions.

Static architectures rely on domain-specific knowledge and predefined rules to establish fixed collaboration channels. This approach ensures that interactions align with rigid requirements, often leveraging sequential connections (chaining) where the output of one agent (y_i) feeds into the next (y_{i+1}) alongside the initial input (x_{collab}):

$$y_{i+1} = y_i \parallel x_i \parallel x_{collab}$$

By utilizing fixed workflows, systems like MapCoder (Islam *et al.*, 2024) (program synthesis), MACRec (Wang *et al.*, 2024) (recommendations), and specialized Literary Translation frameworks like Wu *et al.* (2025) can emulate professional human pipelines. While highly

effective for predictable tasks involving recall, planning, and debugging, these architectures lack the flexibility to adapt to novel or evolving task requirements outside their predefined schema.

In contrast, **Dynamic Architectures** are designed for high adaptability, utilizing Management Agents or adaptive mechanisms to assign roles and define collaboration channels in real-time. This allows the system to respond to the unique nuances of each input rather than following a static script. A prominent implementation of this flexibility is the graph-based orchestration mechanism that an Orchestrator agent dynamically constructs a Directed Acyclic Graph (DAG) where nodes represent discrete tasks and edges define their functional dependencies (Gabriel *et al.*, 2024). This structure enables the system to execute sub-tasks in a optimized parallel or sequential order as dictated by the input. By employing a Delegator agent to consolidate results from all completed nodes, dynamic architectures significantly enhance responsiveness and scalability for complex, multi-step queries that require a non-linear execution sequence.

The analysis of these paradigms reveals that there is no single, unifying definition or classification for Multi-Agent Systems (MAS). Instead, a system's identity is fluid, often exhibiting multiple characteristics simultaneously based on specific architectural requirements. By mapping popular frameworks to these theoretical models, we can better understand how theoretical "backbones" manifest in practical engineering.

As illustrated in the table below, example design patterns in the agentic ecosystem can be cross-referenced against the previously discussed classifications:

Pattern	**AutoGen Group Chat**	**LangGraph / AutoGen Graph**	**CrewAI**
Collaboration	Cooperative	Cooperative	Cooperative
Strategy	Role-based	Model-based	Role-based
Structure	Centralized	Decentralized	Hierarchical
Coordination	Dynamic	Dynamic	Static/Dynamic
Topology (Kim *et al.*, 2025)	Hybrid	Independent	Centralized

Table 2.1 Mapping of Multi-Agent patterns to theoretical classifications.

The fluidity of these classifications is best exemplified by AutoGen's Group Chat pattern (Microsoft, 2024), which is cooperative and role-based but fluctuates between centralized and decentralized structures depending on the presence of a manager agent. While Kim *et al.* (2025) would classify this as a Hybrid topology, its orchestration remains inherently

Dynamic as turn-taking is determined in real-time. Similarly, systems built on LangGraph or AutoGen Graph patterns are characterized by dynamic orchestration, yet they may operate under an Independent communication topology when agents interact through a shared state rather than direct messaging. Ultimately, Derouiche *et al.* (2025) highlights the features of frameworks like LangChain, LangGraph, AutoGen, and CrewAI that can be used to build systems exhibiting a variety of the characterizations discussed.

2.3.2 Shift Towards Structured But Dynamic Orchestration

This paradigm bridges the gap between deterministic control and agentic flexibility by employing explicitly defined control flows that remain highly adaptable rather than following a static, linear script. By hybridizing structured logic with real-time decision-making, these frameworks ensure that multi-agent interactions are both architecturally grounded and responsive to evolving task requirements. To explore the shift toward structured yet dynamic orchestration, we analyze how recent frameworks transition from static chains to complex, topology-aware coordination. These works move beyond simple message passing, instead treating the multi-agent interaction space as a programmable and adaptable graph.

The **Graph of Agents (GoA)** framework (Joo *et al.*, 2025), for instance, operates through a layered information fusion architecture where a query is decomposed and processed in parallel by an initial tier of agents. These preliminary outputs are then funneled into subsequent layers of aggregator agents that resolve contradictions and synthesize a unified response. The mechanical core of GoA is its topology-aware scoring system, which assigns dynamic weights to each agent's contribution during the fusion process, ensuring that the most rational and contextually relevant reasoning paths are prioritized in the final output.

Advancing this adaptability, **DyTopo** (Lu *et al.*, 2026) introduces a highly specialized semantic routing mechanism that reconfigures the communication network for every unique task. Unlike frameworks with fixed edges, DyTopo treats the multi-agent system as a latent pool of capabilities, employing a Topology Manager to analyze the semantic intent of a query and predict the optimal functional dependencies between agents. By calculating the semantic similarity and task-relevance of potential agent pairings, the system dynamically activates or prunes communication edges in real-time. This precision routing eliminates the "noise" and

redundant token usage typical of all-to-all broadcast models, ensuring that information only flows through edges that provide measurable utility to the sub-task at hand.

In the context of large-scale coordination, **MacNet** (Qian *et al.*, 2025) organizes agents into specialized topological neighborhoods within a Directed Acyclic Graph (DAG). This framework utilizes a structural routing protocol that directs the flow of state information based on the hierarchical and functional position of each agent in the network. A key technical feature of MacNet is its use of topological persistence, where the system records and optimizes high-performing routing paths across the DAG over time, effectively allowing the architecture to "learn" more efficient coordination patterns for recurring problem types. This bridges the gap between the rigid constraints of a central orchestrator and the chaotic overhead of purely decentralized debate.

Finally, most recently Ben Khaled and Monticolo (2026) introduces **G2CP (Graph To Communication Protocol)** framework functions as an orchestration compiler that separates the high-level task logic from the execution-level interaction rules. It begins by generating a graph-based plan of the user's goal and subsequently translates that graph into a formal communication protocol a deterministic set of "rules of engagement" that define the specific states, turn-taking logic, and data schemas required for the agents to collaborate. By decoupling the "what" (the graph) from the "how" (the protocol), G2CP allows for a hybrid state where the global objective is structurally anchored, but the local agent interactions remain fluid and adaptive to real-time feedback. Collectively, these advancements highlight a move toward topology-aware intelligence, where the programmable geometry of the agent network becomes as critical to success as the underlying language models.

2.3.3 Extending KG Driven Discovery to MAS

SAP an Enterprise application software company utilizes Knowledge Graphs (KGs) for API discovery by enriching raw API specifications with semantic metadata and business-process context, moving orchestration beyond the limitations of hard-coded scripts (DeepLearning.AI, 2024). This implementation demonstrates that KGs serve as a foundational "source of truth" for the dynamic, deterministic sequencing of complex tasks. By traversing the ontological relationships within the graph, an orchestrator can perform real-time path-finding to

resolve multi-step goals based on verifiable functional dependencies rather than probabilistic generative paths.

This proves that Knowledge Graphs are highly effective for the dynamic sequencing of diverse data types and tasks, ensuring that every transition in a multi-hop workflow is factually grounded and logically valid. Our proposed system seeks to adopt this approach for multi-agent orchestration, leveraging the KG to provide the structural rigor and "Just-in-Time" adaptability required for autonomous, enterprise-grade coordination.

2.4 Conclusion

The analysis of current literature reveals a critical trade-off between architectural rigor and operational flexibility. While graph-based frameworks like MacNet and Graph of Agents (GoA) provide necessary structural constraints through DAGs and layered topologies, their primary gap lies in a tendency to rely on predefined sequencing for specific workflows. By requiring the explicit mapping of node transitions or hierarchical tiers, these systems can become brittle or lose their truly "dynamic" nature when faced with tasks that deviate from their established schema. Similarly, even in more adaptive protocols like G2CP and DyTopo, the logic used to traverse these structures often fluctuates between rigid hard-coding and probabilistic inference.

Future study must address the integration of deterministic symbolic grounding that allows for real-time adaptability without requiring exhaustive manual mapping for every possible scenario. By adopting a Knowledge Graph-based orchestration framework, the proposed system treats the KG as a fluid "source of truth" rather than a fixed flowchart. In this paradigm, the Knowledge Graph serves as an ontological map where nodes represent agentic capabilities and edges define their formal functional dependencies. This shift allows the orchestrator to dynamically query the KG to construct valid, context-specific execution paths on the fly. This ensures that the system's "backbone" remains factually grounded in a verifiable symbolic structure, providing the architectural rigor of a DAG with the unscripted flexibility required for complex, autonomous coordination.

CHAPTER 3

METHODOLOGY AND SYSTEM DESIGN

3.1 Introduction

This chapter details the methodology and system design that guided the development of Mako, a novel agent orchestration framework anchored by a knowledge graph. Designing a system capable of managing autonomous agents requires moving beyond simple linear programming; it necessitates an architecture that can handle ambiguity, manage state dynamically, and route tasks intelligently. To that end, this chapter outlines the research design and the architectural framework that empower Mako to decompose complex, ambiguous user goals, evaluate agent capabilities, and execute tasks in a highly coordinated, asynchronous manner.

Furthermore, this section presents the structural design of the Neo4j-backed knowledge graph, which serves a dual purpose as both the system's shared memory and its state management layer. The interaction protocols between specialized agents are explored, detailing how decoupled communication prevents system bottlenecks. Finally, the chapter outlines the specific tools and technologies selected for implementation, maps out the step-by-step system workflows, and defines the rigorous evaluation metrics utilized to assess the framework's performance, task completion accuracy, and overall orchestration efficiency.

3.2 Research Design

This study adopts a system development and design science research approach, focusing heavily on the architectural engineering, implementation, and empirical evaluation of a multi-agent orchestration framework. The primary motivation driving this methodological choice is the inherent limitation of single Large Language Model (LLM) deployments. While individual LLMs possess vast generalized knowledge, they frequently struggle to handle complex, multi-step, or highly specialized problems reliably without losing context or hallucinating.

To resolve this, the core philosophy of the Mako framework is to decentralize execution while centralizing orchestration, utilizing a Knowledge Graph (KG) as a persistent shared memory and relationship mapper. Traditional programmatic workflows rely on rigid, hard-coded

pathways that fail when presented with edge cases. In contrast, Mako is designed as a dynamic, graph-based architecture. When a user submits a natural language goal, the system does not force it through a predefined script; instead, it dynamically decomposes the objective into a Directed Acyclic Graph (DAG) of discrete tasks. It then maps these tasks to the optimal agents based on a real-time assessment of their capabilities, availability, and trust levels.

This graph-centric approach was selected because knowledge graphs naturally and intuitively model the complex, many-to-many relationships inherent in a workforce of AI agents. By externalizing the state management and relationship tracking to a robust graph database (Neo4j), the individual agents can remain entirely stateless. They wake up, execute their specific task using the context provided, and shut down. Meanwhile, the central orchestrator maintains a persistent, global visibility over task dependencies, agent workloads, and the evolving state of the user's overarching goal.

3.3 System Architecture

To ensure scalability, maintainability, and the flexibility to seamlessly integrate new agent capabilities in the future, the architecture of the Mako orchestration framework is strictly modular. The design separates the cognitive planning layer, the routing logic, the execution environment, and the data storage mechanisms. This separation of concerns is realized through five primary interconnected components:

1. **The User Interface and Entry Point:** The production entry point of the system operates via an interactive Command Line Interface (CLI). This component is responsible for capturing the initial state, booting the connection to the knowledge graph, and capturing natural language goals from the user. It also serves as the real-time feedback loop, translating backend graph node statuses, orchestration decisions, and agent memory logs into formatted, human-readable output.

2. **The Orchestrator Layer:** Acting as the central nervous system of the framework, the Orchestrator manages the complete lifecycle of a user's goal. It does not execute tasks itself; rather, it coordinates the flow of information between the planner, the reasoning engine, and the executor. It monitors the DAG of tasks, ensuring that dependencies are strictly respected and that no task is triggered before its prerequisite data is available in the shared memory.

3. **The Planner Component:** The Planner is the strategic core of the system. It is tasked with ingesting complex, high-level user goals and applying cognitive breakdown strategies to decompose them into discrete, manageable sub-tasks. During this phase, it generates the necessary `Task` nodes and formally establishes the `DEPENDS_ON` relationships that dictate the execution timeline.

4. **The Reasoning and Routing Engine:** Once tasks are generated, the Reasoning Engine evaluates them against the current pool of active agents. It translates task requirements into complex graph queries, scanning the Neo4j database to match tasks to agents. This matching process is multidimensional, taking into account the required capabilities, the trust thresholds necessary for the task, and the current operational load of the available agents to prevent bottlenecks.

5. **The Executor Layer:** The Executor is the operational arm of the framework. It handles the instantiation of tasks by formatting precise prompts, injecting the necessary historical context retrieved from the knowledge graph, and making the API calls to the underlying LLM providers. It also handles error catching and output validation before passing the results back to the Orchestrator.

6. **The Knowledge Graph (Data Layer):** Operating entirely in the background, the Neo4j database acts as the single source of truth for the entire framework. It persistently stores all entities—ranging from Agent profiles to discrete Tasks and Capabilities—and maps their dynamic, shifting relationships in real-time as the system operates.

3.4 Knowledge Graph Design

The integration of a formalized Knowledge Graph is the defining, innovative characteristic of the Mako orchestration framework. In traditional multi-agent systems, agents often pass state back and forth through complex, fragile message queues or bloated JSON objects. In Mako, the knowledge graph serves simultaneously as the system's routing directory and its shared memory space. The schema was specifically engineered to map the ontology of an AI workforce, treating agents, skills, and memories as interconnected nodes.

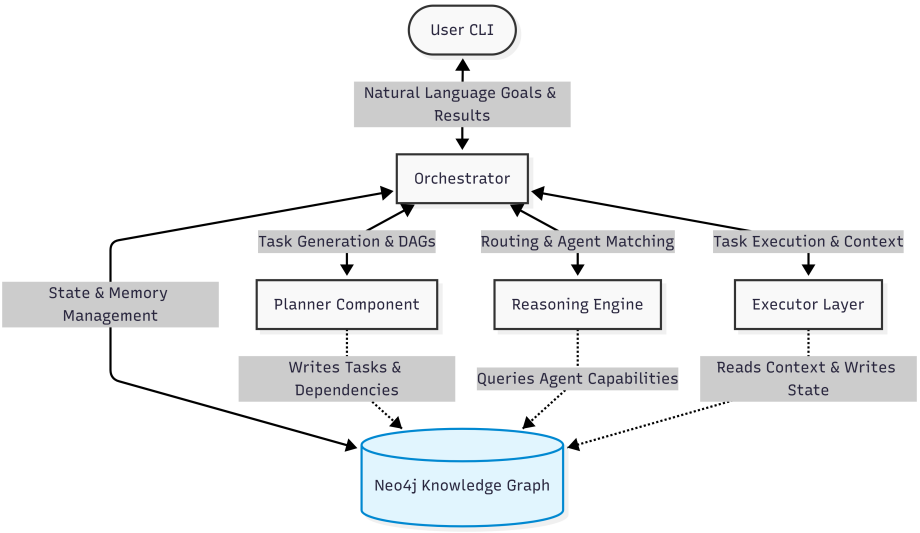

Figure 3.1 Overall system architecture showing the interaction between the User CLI, Orchestrator, Planner, Executor, Reasoning Engine, and Neo4j Knowledge Graph (Authors' own construct).

3.4.1 Graph Structure: Nodes

The graph database utilizes specific, typed node labels to categorize the entities operating within the system. This strict typing allows the Reasoning Engine to filter and query the database with high efficiency.

Table 3.1 Knowledge Graph Node Types

Node Label	Description
Agent	Represents an autonomous AI actor within the system, ranging from orchestrators to specialized worker agents.
Capability	Defines specific, actionable skills an agent possesses, such as `web_search`, `data_analysis`, or `summarization`.
Task	Represents discrete, actionable units of work generated dynamically by the Planner Component.
State	Functions as the memory storage, holding agent outputs and contextual data for future retrieval.
Message	Acts as an execution log, recording inter-agent communications, system alerts, and error states.
Constraint	Represents operational boundaries or rules governing agent behavior, such as specific safety or trust limitations.
Pipeline	Represents the macro-level view of a multi-step execution workflow, grouping related tasks together.

3.4.2 Graph Structure: Relationships (Edges)

Nodes in isolation offer little value; the true power of the framework lies in the edges connecting them. To enable the Reasoning Engine to make intelligent, context-aware routing decisions, the graph relies on strictly defined directional relationships.

Table 3.2 Knowledge Graph Relationship Types

Relationship Type	**Source Node → Target Node**	**Description**
HAS_CAPABILITY	Agent → Capability	Indicates that an agent is equipped and authorized to perform a specific skill.
REQUIRES_CAPABILITY	Task → Capability	Defines the prerequisite skill an agent must have to successfully execute the mapped task.
ASSIGNED_TO	Agent → Task	A dynamic edge indicating the current active workload and assignment of a specific agent.
DEPENDS_ON	Task → Task	Enforces strict execution ordering; ensuring a task cannot be initiated until its dependency is marked complete.
HOLDS_STATE	Agent → State	Links an agent to its historical context, allowing it to "remember" past outputs and decisions.
GOVERNED_BY	Agent → Constraint	Applies strict security protocols, operational limits, or trust boundaries to specific agents.

The state of these edges is highly fluid. When the Planner decomposes a new user goal, it creates a web of new `Task` nodes connected by `REQUIRES_CAPABILITY` edges. Milliseconds later, the Reasoning engine traverses the graph to find `Agent` nodes connected via `HAS_CAPABILITY` to those exact skills. Upon execution, the Orchestrator permanently records the interaction by drawing `ASSIGNED_TO` and `HOLDS_STATE` edges, creating an auditable trail of how the system solved the problem.

3.5 Agent Design

Mako leverages a heterogeneous multi-agent design philosophy. Rather than utilizing a single, massive prompt to force an LLM to act as a generalist, the framework instantiates different agents with highly specialized roles, distinct operational capabilities, and varying

trust levels. This mimics human organizational structures, where specialists handle specific tasks under the guidance of a project manager.

3.5.1 Agent Roles and Capabilities

To validate the framework, the system is seeded with a baseline roster of specialized agents designed to handle diverse workflows:

1. **The Orchestrator Agent (`orch-001`):** Acting as the primary manager, this agent operates with a maximum trust level of 1.0. It possesses macro-planning capabilities and is strictly responsible for overseeing the execution of the user's goal, rather than doing the granular work itself.
2. **The Research Agent (`agent-r1`):** Specialized in broad information retrieval and external data validation. It holds the `web_search` capability and operates with a trust level of 0.8, reflecting the inherent unpredictability of querying external web sources.
3. **The Summarization Agent (`agent-s1`):** Engineered to synthesize large volumes of text and extract key metrics. It holds the `summarization` capability and operates with a trust level of 0.8, ensuring it relies strictly on provided context rather than hallucinating external facts.
4. **The Coder Agent (`agent-c1`):** A highly specialized actor designed for technical tasks, script generation, and code execution. Because executing arbitrary code carries inherent system risks, it holds the `code_execution` capability but operates under a stricter trust level of 0.75.

3.5.2 Agent Interaction Model

A critical design choice in the Mako framework is that agents do not communicate via direct point-to-point APIs. Direct communication often leads to tightly coupled systems that break when a single agent fails or times out. Instead, Mako agents communicate entirely through the Knowledge Graph.

For example, when the `Research Agent` completes a targeted web search, it does not send the data directly to the next agent. Instead, its output is logged into a `State` node and linked to the overarching task graph. When the Orchestrator subsequently wakes up

the `Summarization Agent`, the Executor queries the graph, retrieves that `State` node, and safely injects the data into the Summarizer's context window. This creates a highly decoupled, asynchronous interaction model that is resilient to timeouts, scalable, and easy to audit.

3.6 Orchestration Mechanism

The orchestration mechanism is the underlying procedural logic—the system's heartbeat—that dictates exactly how user goals transition from abstract concepts to completed outputs.

3.6.1 Task Allocation and Decision Flow

The lifecycle of a goal within Mako follows a rigorously defined pipeline:

1. **Goal Ingestion:** The system captures a natural language request from the user (e.g., "Search for recent AI orchestration papers and summarize their key findings").
2. **Task Decomposition:** The Planner intercepts the request, applying LLM reasoning to break it into a logical DAG of subtasks. It identifies that this requires two distinct phases:
   - *Task 1:* "Search AI orchestration papers" (Requires: `web_search`, Output Key: `papers`)
   - *Task 2:* "Summarize the retrieved papers" (Requires: `summarization`, Depends On: Task 1)
3. **Dependency Resolution:** The Orchestrator continually queries the graph for tasks that have no outstanding `DEPENDS_ON` relationships. It identifies Task 1 as ready for execution.
4. **Intelligent Agent Routing:** The Reasoning component takes Task 1 and queries the Neo4j graph for any active agents possessing the `web_search` capability and an adequate trust score. It successfully matches the task to `agent-r1`.
5. **Execution and Aggregation:** The Executor packages Task 1 into a prompt and passes it to `agent-r1`. Upon completion, the outputs are captured, stored in a local

accumulator (`accumulated_outputs["papers"]`), and permanently written to a new `State` node in the graph.

6. **Workflow Continuation:** The Orchestrator marks Task 1 as successfully completed, which satisfies the dependency for Task 2, unlocking it. Task 2 is then routed to `agent-s1`, with the Executor automatically fetching the `papers` output from the graph to provide context.

7. **Final Aggregation:** Once the Orchestrator verifies that all nodes in the DAG have reached a completed state, the final synthesized output is formatted. It is then returned to the user alongside rich metadata, including execution duration, agent load statistics, and an aggregate confidence score.

3.7 Implementation Tools and Technologies

Moving from conceptual design to functional software required a modern, robust technology stack. The tools chosen for the Mako framework prioritize graph processing efficiency, seamless AI integration, and high-performance execution.

Table 3.3 Development Tools and Technologies

Tool / Technology	Purpose / Justification
Python 3.10+	Serves as the primary backend language. Python was selected for its unparalleled AI/Machine Learning ecosystem, extensive library support, and native ability to handle complex asynchronous data manipulation.
Neo4j	The core graph database utilized for the Knowledge Graph. Neo4j was chosen over traditional relational databases due to its native Cypher query language, which is specifically optimized for deep pathfinding and multi-hop relationship mapping.
Pydantic	Implemented as the schema and validation layer. Pydantic enforces strict data contracts between the unpredictable outputs of the LLMs and the rigid requirements of the Python backend, preventing malformed data from crashing the system.
LLM APIs (OpenAI/Anthropic)	These APIs act as the cognitive engines for the agents. They power the higher-order reasoning required by the Planner, the Executor, and the individual specialized workers.
Neo4j Python Driver	The bridge component that facilitates seamless, secure execution of Cypher queries, pushing updates from the orchestration backend directly into the graph database.

3.8 System Workflow

The end-to-end operation of Mako is not left to chance; it follows a strict, predictable execution pipeline designed to ensure maximum reliability, traceability, and error recovery.

1. **System Boot Sequence (`main.py` → `boot()`):** Upon startup, the framework establishes a secure connection to the Neo4j instance. It runs a health check, verifies the graph schema, and if the database is found empty, automatically seeds the environment with the initial required agents, baseline capabilities, and systemic constraints.
2. **Input Capture:** The CLI enters a listening state, awaiting user input.
3. **Cognitive Planning Phase (`planner.py`):** The user's input is routed to the planning LLM accompanied by a strict system prompt. The LLM processes the goal and returns a highly structured JSON object representing the DAG of required tasks.
4. **Graph Population (`orchestrator.py`):** The system parses the JSON and instantiates the tasks as physical nodes within the Neo4j graph, establishing the critical `REQUIRES_CAPABILITY` and `DEPENDS_ON` edges that will guide execution.
5. **The Asynchronous Execution Loop:** The core loop begins:
   - The system scans for unlocked, available tasks.
   - The Reasoning module (`reasoning.py`) routes the task to the optimal agent.
   - The Executor module (`executor.py`) runs the task and captures the text and metadata outputs.
   - The Database module (`database.py`) updates the task state to "completed" in the graph and logs the new `State` node.
6. **Result Delivery:** Once the loop concludes, the orchestrator formats the aggregated results into a human-readable format and renders them to the CLI, providing the user with both the answer and the metadata detailing how the system arrived at it.

3.9 System Evaluation and Testing

To ensure the proposed Mako framework successfully meets its theoretical objectives, a rigorous, multi-tiered evaluation methodology was established. The system is continuously

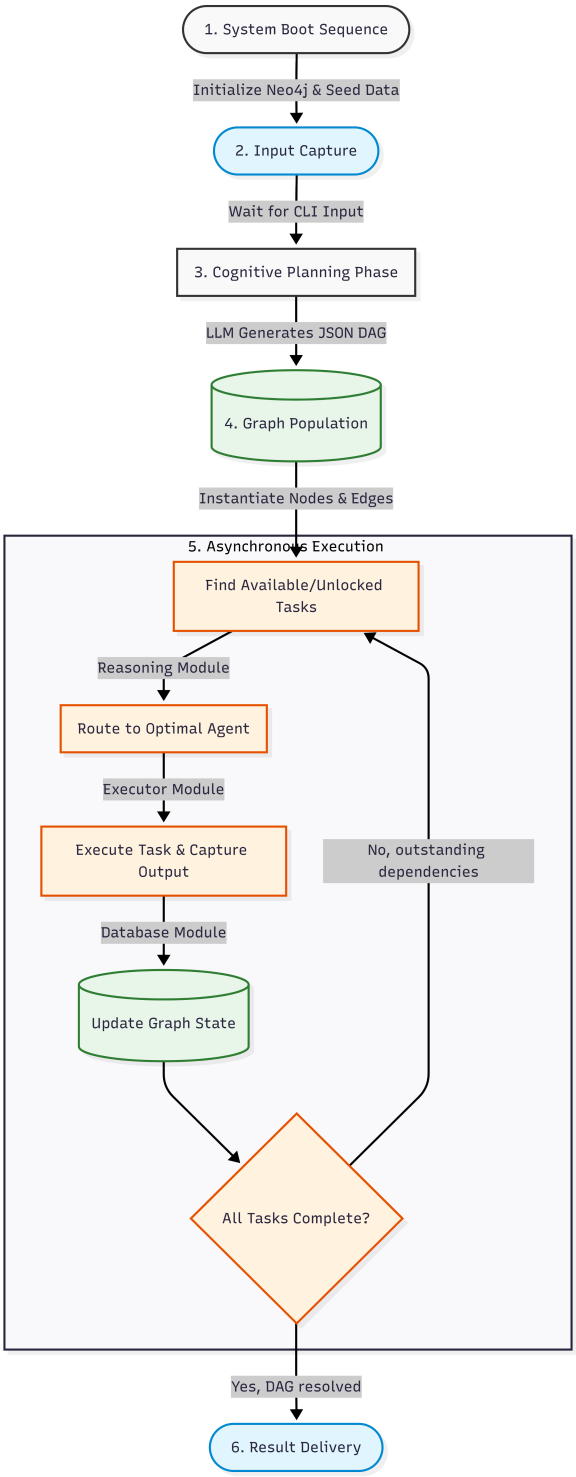

Figure 3.2 System workflow flowchart showing the sequence from system boot through the execution loop to result delivery (Authors' own construct).

evaluated based on its functional capability to decompose complex tasks accurately, as well as the operational efficiency and safety of its multi-agent interactions.

3.9.1 Evaluation Metrics

To move beyond subjective observation, the following quantifiable metrics are utilized to benchmark the performance of the framework:

Table 3.4 System Evaluation Metrics

Metric	Description	Target / Success Criteria
Task Completion Rate	The percentage of user goals that are successfully decomposed, routed, and fully executed without triggering a systemic failure or timeout.	$> 90\%$ completion rate on standard, multi-step user requests.
Routing Accuracy	The frequency with which the Reasoning engine successfully selects the correct agent for a specific task based solely on the required capabilities.	100% adherence to capability constraints (zero misrouted tasks).
System Response Time	The total end-to-end duration from the moment of user prompt submission to the generation of the final synthesized result.	Minimized systemic overhead, ensuring the primary bottleneck is solely the unavoidable LLM network latency.
Graph Integrity	A measure of database health; checking the consistency of the Neo4j state following complex execution pipelines to ensure no orphaned nodes or logic breaks exist.	0% orphaned states or broken dependency chains during execution logging.

3.9.2 Testing Procedures

To capture data against the aforementioned metrics, the system undergoes three distinct phases of testing:

1. **Unit Testing:** Individual software components (the Planner, the Reasoning Engine, the Executor) are tested in isolation. To prevent API latency from skewing the results, these tests utilize mocked, static LLM responses. This ensures deterministic behavior and verifies that the underlying Python logic is mathematically sound.

2. **Integration Testing:** The critical interaction between the Python orchestration layer and the Neo4j database is tested under load. These tests verify that node creation, complex edge mapping, and real-time state updates occur flawlessly without deadlocks or race conditions.

3. **End-to-End Functional Testing:** The fully assembled system is subjected to complex, highly ambiguous, multi-layered prompts (e.g., "Analyze current artificial intelligence

tech trends, write a comprehensive summary report, and format the final output as a valid HTML document"). This holistic testing evaluates the system's ability to plan, route, and execute a multi-agent workflow seamlessly from start to finish in a production-like environment.

CHAPTER 4

CHAPTER TITLE HERE

4.1 FIRST SECTION TITLE GOES HERE

4.1.1 Your subsection

CHAPTER 5

CONCLUSION AND RECOMMENDATION

5.1 Conclusion

5.2 Recommendations

REFERENCES

Albada, M. (2025a). “Title of Chapter”. In: *Building Applications with AI Agents*. O’Reilly Media, p. 163.

— (2025b). “Title of Chapter”. In: *Building Applications with AI Agents*. O’Reilly Media, pp. 116 –123.

Amayuelas, A., Sain, J., Kaur, S., and Smiley, C. (2025). *Grounding LLM Reasoning with Knowledge Graphs*. arXiv: 2502.13247 [cs.CL]. URL: https://arxiv.org/abs/2502.13247.

Anthropic (2024). *Effective Context Engineering for AI Agents*. Accessed: 2026-03-31. URL: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Ben Khaled, K. and Monticolo, D. (2026). “G2CP: A Graph-Grounded Communication Protocol for Verifiable and Efficient Multi-Agent Reasoning”. In: Accessed: 2026-04-01. arXiv: 2602.13370 [cs.MA]. URL: https://arxiv.org/abs/2602.13370.

Catherine, R. and Cohen, W. W. (2016). “Personalized Recommendations Using Knowledge Graphs: A Probabilistic Logic Programming Approach”. In: *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys ’16)*. New York, NY, USA: Association for Computing Machinery, pp. 325–332. DOI: 10.1145/2959100.2959131. URL: https://doi.org/10.1145/2959100.2959131.

Cheng, J., Su, T., Yuan, J., He, G., Liu, J., Tao, X., Xie, J., and Li, H. (2025). *Chain-of-Thought Prompting Obscures Hallucination Cues in Large Language Models: An Empirical Evaluation*. arXiv: 2506.17088 [cs.CL]. URL: https://arxiv.org/abs/2506.17088.

DeepLearning.AI (2024). *Knowledge Graphs for AI Agent: API Discovery*. Instructors: Pavithra G K and Lars Heling. Accessed: 2026-04-01. DeepLearning.AI. URL: https://www.deeplearning.ai/short-courses/knowledge-graphs-for-ai-agent-api-discovery/.

Derouiche, H., Brahmi, Z., and Mazeni, H. (2025). *Agentic AI Frameworks: Architectures, Protocols, and Design Challenges*. arXiv: 2508.10146 [cs.AI]. URL: https://arxiv.org/abs/2508.10146.

Elementum Team (2026). “Deterministic vs. Probabilistic AI: The Enterprise Guide to Building Workflows That Scale”. In: *Elementum AI Blog*. Accessed: 2026-03-26. URL: https://www.elementum.ai/blog/deterministic-vs-probabilistic-ai.

Gabriel, A. G., Ahmad, A. A., and Jeyakumar, S. K. (Oct. 2024). *Advancing Agentic Systems: Dynamic Task Decomposition, Tool Integration and Evaluation using Novel Metrics and Dataset*. arXiv: 2410.22457 [cs.AI]. URL: https://arxiv.org/abs/2410.22457.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., and Zhang, X. (2024). *Large Language Model based Multi-Agents: A Survey of Progress and Challenges*. arXiv: 2402.01680 [cs.CL]. URL: https://arxiv.org/abs/2402.01680.

Hagendorff, T., Fabi, S., and Kosinski, M. (2023). “Human-like intuitive behavior and reasoning biases emerged in large language models but disappeared in ChatGPT”. In: *Nature Computational Science* 3.10, pp. 833–838. DOI: 10.1038/s43588-023-00529-1.

Hogan, A., Blomqvist, E., Cochez, M., D’amato, C., Melo, G. D., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., Ngomo, A.-C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., and Zimmermann, A. (July 2021). “Knowledge Graphs”. In: *ACM Computing Surveys* 54.4, 1–37. ISSN: 1557-7341. DOI: 10.1145/3447772. URL: http://dx.doi.org/10.1145/3447772.

Islam, M. A., Ali, M. E., and Parvez, M. R. (2024). *MapCoder: Multi-Agent Code Generation for Competitive Problem Solving*. arXiv: 2405.11403 [cs.CL]. URL: https://arxiv.org/abs/2405.11403.

Jian, Y. (2025). “Conceptual Foundations of LLM-Powered Agents: From Language Processing to Autonomous Reasoning”. In: *Scientific Journal of Intelligent Systems Research* 7.4, 29–35. DOI: 10.54691/atmxvb43. URL: http://dx.doi.org/10.54691/atmxvb43.

Jiang, X., Xu, C., Shen, Y., Sun, X., Tang, L., Wang, S., Chen, Z., Wang, Y., and Guo, J. (2025). *On the Evolution of Knowledge Graphs: A Survey and Perspective*. arXiv: 2310.04835 [cs.AI]. URL: https://arxiv.org/abs/2310.04835.

Joo, T., Ishida, S., Sosnovik, I., Lim, B., Rezaei-Shoshtari, S., Gaier, A., and Giaquinto, R. (2025). *Graph of Agents: Principled Long Context Modeling by Emergent Multi-Agent Collaboration*. arXiv: 2509.21848 [cs.LG]. URL: https://arxiv.org/abs/2509.21848.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D., *et al.* (2020). "Scaling Laws for Neural Language Models". In: *arXiv:2001.08361*. arXiv: 2001.08361 [cs.LG].

Kau, A., He, X., Nambissan, A., Astudillo, A., Yin, H., and Aryani, A. (2024). *Combining Knowledge Graphs and Large Language Models*. arXiv: 2407.06564 [cs.CL]. URL: https://arxiv.org/abs/2407.06564.

Kejriwal, M., Knoblock, C. A., and Szekely, P. (2021). *Knowledge Graphs: Fundamentals, Techniques, and Applications*. Cambridge, Massachusetts: MIT Press, pp. 50–65. ISBN: 9780262045094.

Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A. A., Yan, Y., Zhang, Z., Zhuang, Y., Malhotra, M., Liang, P. P., Park, H. W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., and Liu, X. (2025). *Towards a Science of Scaling Agent Systems*. arXiv: 2512.08296 [cs.AI]. URL: https://arxiv.org/abs/2512.08296.

LangChain Inc. (2024). *LangGraph Graph API*. https://docs.langchain.com/oss/python/langgraph/graph-api. Accessed: 2026-03-26.

Latitude (2024). *How Task Complexity Drives Error Propagation in LLMs*. Accessed: 2026-03-31. URL: https://latitude.so/blog/how-task-complexity-drives-error-propagation-in-llms.

Li, J., Zhang, Q., Yu, Y., Fu, Q., and Ye, D. (2024). "More Agents Is All You Need". In: *Transactions on Machine Learning Research*. URL: https://openreview.net/forum?id=bgzUSZ8aeg.

Liu, M. X., Liu, F., Fiannaca, A. J., Koo, T., Dixon, L., Terry, M., and Cai, C. J. (May 2024). ""We Need Structured Output": Towards User-centered Constraints on Large Language Model Output". In: *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. CHI '24. ACM, 1–9. DOI: 10.1145/3613905.3650756. URL: http://dx.doi.org/10.1145/3613905.3650756.

Lu, Y., Hu, Y., Zhao, X., and Cao, J. (2026). *DyTopo: Dynamic Topology Routing for Multi-Agent Reasoning via Semantic Matching*. arXiv: 2602.06039 [cs.AI]. URL: https://arxiv.org/abs/2602.06039.

Menshawy, A. and Fahmy, M. (2025). *LLMs in Enterprise: Design Strategies for Large Language Model Development, Design Patterns and Best Practices*. Packt Publishing, p. 71. ISBN: 978-1836203070.

Microsoft (2024). *Group Chat Design Pattern*. Accessed: 2026-04-01. Microsoft. URL: https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/design-patterns/group-chat.html.

Nath, A. (2025). "A Practical Perspective on Orchestrating AI Agent Systems with DAGs". In: *Medium*. Accessed: 2026-03-26. URL: https://medium.com/@arpitnath42/a-practical-perspective-on-orchestrating-ai-agent-systems-with-dags-c9264bf38884.

Niu, M., Li, H., Shi, J., Haddadi, H., and Mo, F. (2024). *Mitigating Hallucinations in Large Language Models via Self-Refinement-Enhanced Knowledge Retrieval*. arXiv: 2405.06545 [cs.CL]. URL: https://arxiv.org/abs/2405.06545.

Nizar, F., Lumer, E., Gulati, A., Basavaraju, P. H., and Subbiah, V. K. (2025). *Agent-as-a-Graph: Knowledge Graph-Based Tool and Agent Retrieval for LLM Multi-Agent Systems*. arXiv: 2511.18194 [cs.CL]. URL: https://arxiv.org/abs/2511.18194.

Noy, N., Gao, Y., Jain, A., Narayanan, A., Patterson, A., and Taylor, J. (2019). "Industry-Scale Knowledge Graphs: Lessons and Challenges". In: *Communications of the ACM* 62.8, pp. 36–43. DOI: 10.1145/3331166. URL: https://doi.org/10.1145/3331166.

Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., and Wu, X. (July 2024). "Unifying Large Language Models and Knowledge Graphs: A Roadmap". In: *IEEE Transactions on Knowledge and Data Engineering* 36.7, 3580–3599. ISSN: 2326-3865. DOI: 10.1109/tkde.2024.3352100. URL: http://dx.doi.org/10.1109/TKDE.2024.3352100.

Parisi, A., Zhao, Y., and Fiedel, N. (2022). *TALM: Tool Augmented Language Models*. arXiv: 2205.12255 [cs.CL]. URL: https://arxiv.org/abs/2205.12255.

Qian, C., Xie, Z., Wang, Y., Liu, W., Zhu, K., Xia, H., Dang, Y., Du, Z., Chen, W., Yang, C., Liu, Z., and Sun, M. (2025). *Scaling Large Language Model-based Multi-Agent Collaboration*. arXiv: 2406.07155 [cs.AI]. URL: https://arxiv.org/abs/2406.07155.

Rana, A. and Kumar, G. (2025). *Model-First Reasoning LLM Agents: Reducing Hallucinations through Explicit Problem Modeling*. arXiv: 2512.14474 [cs.AI]. URL: https://arxiv.org/abs/2512.14474.

Sanmartin, D. (2024). *KG-RAG: Bridging the Gap Between Knowledge and Creativity*. arXiv: 2405.12035 [cs.AI]. URL: https://arxiv.org/abs/2405.12035.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv: 2302.04761 [cs.CL]. URL: https://arxiv.org/abs/2302.04761.

Shankar, S. (2025). *LangGraph as a DAG: Rethinking Data Pipeline Orchestration*. Medium article. URL: https://medium.com/@srikrishnan.tech/langgraph-as-a-dag-rethinking-data-pipeline-orchestration-f089ccea175b.

Stegeman, J. (2023). *Native vs. Non-Native Graph Database Architecture & Technology*. Accessed: 2026-04-01. Neo4j. URL: https://neo4j.com/blog/native-vs-non-native-graph-technology/.

Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., and Nguyen, H. D. (2025). *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*. arXiv: 2501.06322 [cs.AI]. URL: https://arxiv.org/abs/2501.06322.

Valmeekam, K., Marquez, M., Olmo, A., Sreedharan, S., and Kambhampati, S. (2023). *PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change*. arXiv: 2206.10498 [cs.CL]. URL: https://arxiv.org/abs/2206.10498.

Wang, Y., Pan, Y., Su, Z., Deng, Y., Zhao, Q., Du, L., Luan, T. H., Kang, J., and Niyato, D. (2025). *Large Model Based Agents: State-of-the-Art, Cooperation Paradigms, Security and Privacy, and Future Trends*. arXiv: 2409.14457 [cs.AI]. URL: https://arxiv.org/abs/2409.14457.

Wang, Z., Yu, Y., Zheng, W., Ma, W., and Zhang, M. (July 2024). "MACRec: A Multi-Agent Collaboration Framework for Recommendation". In: *Proceedings of the 47th*

International ACM SIGIR Conference on Research and Development in Information Retrieval. SIGIR 2024. ACM, 2760–2764. DOI: 10.1145/3626772.3657669. URL: http://dx.doi.org/10.1145/3626772.3657669.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2023). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903.

Weng, L. (2023). "LLM-powered Autonomous Agents". In: *Lil'Log*. Accessed: 2026-03-26. URL: https://lilianweng.github.io/posts/2023-06-23-agent/.

Wu, M., Xu, J., Yuan, Y., Haffari, G., Wang, L., Luo, W., and Zhang, K. (2025). *(Perhaps) Beyond Human Translation: Harnessing Multi-Agent Collaboration for Translating Ultra-Long Literary Texts*. arXiv: 2405.11804 [cs.CL]. URL: https://arxiv.org/abs/2405.11804.

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X., Yin, Z., Dou, S., Weng, R., Cheng, W., Zhang, Q., Qin, W., Zheng, Y., Qiu, X., Huang, X., and Gui, T. (2023). *The Rise and Potential of Large Language Model Based Agents: A Survey*. arXiv: 2309.07864 [cs.AI]. URL: https://arxiv.org/abs/2309.07864.

Yang, Y., Peng, Q., Wang, J., Wen, Y., and Zhang, W. (2024). *LLM-based Multi-Agent Systems: Techniques and Business Perspectives*. arXiv: 2411.14033 [cs.AI]. URL: https://arxiv.org/abs/2411.14033.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. (2023a). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. arXiv: 2305.10601 [cs.CL]. URL: https://arxiv.org/abs/2305.10601.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023b). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv: 2210.03629 [cs.CL]. URL: https://arxiv.org/abs/2210.03629.

Zhang, Y., Wang, X., Liang, J., Xia, S., Chen, L., and Xiao, Y. (2024a). *Chain-of-Knowledge: Integrating Knowledge Reasoning into Large Language Models by Learning from Knowledge Graphs*. arXiv: 2407.00653 [cs.CL]. URL: https://arxiv.org/abs/2407.00653.

Zhang, Y., Chen, K., Bai, X., Kang, Z., Guo, Q., and Zhang, M. (Nov. 2024b). "Question-guided Knowledge Graph Re-scoring and Injection for Knowledge Graph Question Answering". In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Y. Al-Onaizan, M. Bansal, and Y.-N. Chen. Miami, Florida, USA: Association for Computational Linguistics, pp. 8972–8985. DOI: 10.18653/v1/2024.findings-emnlp.524. URL: https://aclanthology.org/2024.findings-emnlp.524/.

Zhu, Y., Jin, T., Pruksachatkun, Y., Zhang, A. K., Liu, S., Cui, S., Kapoor, S., Longpre, S., Meng, K., Weiss, R., Barez, F., Gupta, R., Dhamala, J., Merizian, J., Giulianelli, M., Coppock, H., Ududec, C., Kellermann, A., Sekhon, J. S., Steinhardt, J., Schwettmann, S., Narayanan, A., Zaharia, M., Stoica, I., Liang, P., and Kang, D. (2025). "Establishing Best Practices for Building Rigorous Agentic Benchmarks". In: *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. Accessed: 2026-04-01. URL: https://openreview.net/forum?id=E58HNCqoaA.